

volatile关键字作用

volatile 是用于指示编译器，一个变量的值可能会被程序之外的因素改变，因此禁止编译器对该变量进行优化。通常用于多线程编程中，确保线程间共享的变量的可见性和内存一致性。具体作用包括：
防止编译器对该变量进行优化，确保每次访问该变量时，都是直接从内存中读取，而不是使用寄存器缓存。

代码题：两个升序链表合并成一个降序链表

```
#include <stdio.h>
#include <stdlib.h>

struct ListNode {
    int val;
    struct ListNode *next;
};

struct ListNode* create_node(int val) {
    struct ListNode* new_node = (struct ListNode*)malloc(sizeof(struct
ListNode));
    new_node->val = val;
    new_node->next = NULL;
    return new_node;
}

struct ListNode* mergeTwoLists(struct ListNode* l1, struct ListNode* l2) {
    struct ListNode dummy;
    struct ListNode* current = &dummy;
    dummy.next = NULL;

    // 合并两个升序链表
    while (l1 != NULL && l2 != NULL) {
        if (l1->val < l2->val) {
            current->next = l1;
            l1 = l1->next;
        } else {
            current->next = l2;
            l2 = l2->next;
        }
        current = current->next;
    }

    current->next = (l1 != NULL) ? l1 : l2;

    // 将合并后的链表逆序
    struct ListNode* prev = NULL;
    struct ListNode* next = NULL;
    current = dummy.next;

    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
```

```

    }

    return prev;
}

void printList(struct ListNode* head) {
    while (head != NULL) {
        printf("%d -> ", head->val);
        head = head->next;
    }
    printf("NULL\n");
}

void freeList(struct ListNode* head) {
    struct ListNode* temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

int main() {
    struct ListNode* l1 = create_node(1);
    l1->next = create_node(3);
    l1->next->next = create_node(5);

    struct ListNode* l2 = create_node(2);
    l2->next = create_node(4);
    l2->next->next = create_node(6);

    // 合并两个链表并将其逆序
    struct ListNode* merged_list = mergeTwoLists(l1, l2);

    // 打印合并后的链表
    printList(merged_list);

    // 释放链表内存
    freeList(merged_list);

    return 0;
}

```

将无符号 `int` 类型数的二进制数最后一位由 1 改成 0

```

unsigned int num = 5; // 示例值: 101
num = num & (~1);    // 清除最后一位

```

UART 的数据格式？如何保证在侦听 UART 时不会一直判断接收到 0？

起始位（1 位）：表示数据传输的开始，通常为低电平 `0`。

数据位（5 至 9 位）：实际的数据位，可以配置为 5 到 9 位。

奇偶校验位（1 位，可选）：用于检测传输数据的奇偶性。

停止位（1 至 2 位）：表示数据传输的结束，通常为高电平 `1`。

配置硬件中断：当 UART 接收到数据时触发中断，避免主动轮询判断。

设置超时机制：如果长时间未接收到数据，可以认为通信中断或数据错误。

检查起始位：UART 在接收数据前通常会检测到起始位的下降沿，即 `1` 到 `0` 的转换，这可以避免误判接收到的 0。

寄存器长度变小，寻址范围会变小么？

寄存器长度变小会导致寻址范围变小。

寄存器长度：指的是寄存器的宽度（通常以位为单位，如 32 位、64 位）。

寻址范围：通常由寄存器的宽度决定。例如，32 位的寄存器可以寻址的范围是 2^{32} ，即 4GB 地址空间；而 64 位的寄存器可以寻址的范围是 2^{64} 。

如果寄存器长度变小，能表示的地址空间也会相应减小，从而导致可寻址范围变小。

中断的上半部与下半部是如何分离实现的

上半部

上半部是中断处理的第一部分，通常在中断发生时立即执行，以下是其主要特点：

快速响应：上半部的主要任务是快速响应中断并执行一些时间紧迫的操作，例如确认中断源、保存现场信息、清除中断标志等。

在中断上下文执行：上半部在中断上下文中执行，也就是说，它是在处理器中断时运行的。这意味着在执行上半部时，系统无法处理其他中断（除非支持嵌套中断）。

不进行耗时操作：由于上半部在中断上下文中执行且影响系统的整体响应性，因此通常只执行那些必须立即处理的任务，避免任何可能导致延迟的操作，如访问慢速设备或进行复杂计算。

下半部

下半部是中断处理的延续部分，主要用于处理那些可以稍后完成的操作。它的主要特点包括：

延迟执行：下半部的任务可以延迟执行，通常在操作系统恢复到正常调度状态后执行。这意味着它是在内核进程上下文中执行的，不是在中断上下文中。

处理耗时操作：由于下半部不在中断上下文中执行，它可以进行更多的耗时操作

为什么要分离上半部和下半部？

提高响应性：中断上半部处理时间越短，系统对其他中断的响应速度就越快。因此，必须立即执行的操作放在上半部，而可以稍后处理的任务则放在下半部。

降低中断处理对系统的影响：如果所有中断处理都在中断上下文中进行，可能会导致系统其他重要任务被长期阻塞，影响整体性能。通过分离，将较复杂和耗时的任务放到下半部处理，可以减少这种影响

字符驱动的poll机制

wq是一个等待队列，poll机制只是将线程放进wq中，但是未休眠
for会执行两次，第一次查询没有数据，休眠一段时间，第二次查询，没有数据返回，假如有数据按键按下了，在按键中断程序将event=tured; wake_up_interruptible(wq);
我们只用实现挂入队列和返回状态，剩下的内核帮你实现了

app怎么找到驱动?

通过属性(是什么驱动程序)、主设备号、次设备号
某种程度下讲，/dev/driver它的属性会放在一个字符设备链表，sys_open就会在这个链表 里面找chrdevs[主设备号]这一项

自旋锁、信号量和互斥量三个的区别

1. 自旋锁: 自旋锁也和信号量差不多，不过自旋锁获得锁之前会忙等待，cpu无法执行其他的事情，不断的查询这个锁的状态
2. 信号量: 信号量访问的时候就是申请使用，能不能支持多个线程同时访问这个信号资源，要在信号量初始化sem_init的第二个参数设置，然后释放信号量
3. 互斥量: 互斥量就是一个特殊的信号量，互斥只能一个线程持有，一个线程获得锁之后，其他线程只能等待这个线程释放锁

单核和多核情况下的自旋锁的区别

单核情况下: 假如一个任务要获得自旋锁，它就一直查询，但是cpu只有一个，其他的任务它无法执行，那么就无法释放自旋锁，那么就一直卡死
多核情况: 可以正常执行，一个核忙等待，一个核继续处理其他事情

linux 驱动里面的 probe 函数是什么时候被调用以及它的作用

当一个设备被插入到系统中，并且与相应的驱动匹配成功时，内核会调用该驱动的 probe 函数
probe函数的作用：
1. 获取硬件信息，在字符型驱动程序中，一般使用gpiod_get->设置gpio的模式
input/output(gpiod_direction_input/gpiod_direction_output)
我自己的话写驱动的话，我的中断也是在probe函数中申请的。
2. 注册设备device_create

为什么栈从上往下生长

堆是往上涨的，栈是往下涨的，假如栈往上涨的话，那么要确定堆和栈的分界线，每一个程序的堆和栈都一样，所以这个分界线不好确定，所以一个往上涨，一个往下长可以最大化的利用内存的空间